Submission Instructions

Download the file ***Cistagram.cpp*** *from Domjudge contest* **Lab2-User**.
You need to complete this code and submit.
Two kinds of submissions are required:
1. to Domjudge **Lab2-User** and **Lab2-Cistagram** (for yourselves),
2. to LMS (for us to evaluate)
Both Domjudge contest and LMS submission links will close at 3.30pm.
LMS submission is mandatory for attendance and later viva.
Submit your latest file on time even if your code is not complete.
Do not change the file name.
Do not change anything in the NO EDIT ZONE.
Do not change any class name and their public members. However, you are welcome to add/modify private members as per your needs.

Let us have a further simplified Cistagram description wrt the last quiz:

Any person can register as a user with a username she chooses.

If a username already exists, registration fails.

A registered user can login to the app.

After login, a user can create a post by uploading one photo and specifying a caption for it.

A photo is basically a binary 2D matrix, a recorded timestamp, and a tagged location.

Scrolling of posts is very simplified here: it is just displaying all posts one after another, as per their creation order.

User can also delete a post in the following manner: she specifies the index of the post that she wants to delete, and it gets deleted.

Task Description:

The skeleton code already provides the basic structure and command-driven behavior of the program.

Follow the code comments and fill-in the details.

Lab2-P1: Complete Only the User class.

Lab2-P2: Take your Lab2-P1 code and complete the Cistagram class.

Dynamic Memory Allocation

In this task, you need to allocate dynamic memory, for creating photo and user.

In C++, it is done using the **new** operator. Memory is later freed using the **delete** operator.

There is no malloc and free in C++.

Here is one example:

```cpp
#include<iostream>
#include<cstdlib>
using namespace std;

class Cat {
public:
    string name;
    int speed;
    void show() {
        cout<<name<<" "<<speed<<endl;
    }
};

int main() {
    Cat* tom = new Cat{ .name: "Tom", .speed: 0}; // dynamic memory allocation
    tom->show();
    delete tom; // memory free
}
```

Memory leaking programs may get segmentation fault in Domjudge, even if it works locally.

So, give special attention to the implementation of memory freeing methods.